SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	**Assessment:**	Experiments
Weightage:	45%	**Total Marks:**	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	**G LOKESH YADAV**	**Reg. No.:**	**192411061**
Section / Batch:	**2024-CSA**	**Date:**	**10-08-2026**

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct

I G LOKESH YADAV(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Lokesh yadav

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	

5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Course Coordinator Faculty Incharge

1. Register Transfer and ALU Operations

Objective: Design and simulate register transfer operations together with basic arithmetic and logical operations using multiple registers and an ALU.

Theory

Register transfer operations move binary data between CPU registers. The ALU performs arithmetic and logical operations on operands supplied by registers. Typical transfers are R2 ← R1, R3 ← R2, while ALU operations include R3 ← R1 + R2, R3 ← R1 − R2, R3 ← R1 AND R2 and R3 ← R1 OR R2.

Python Simulation

```
R = {"R1": 25, "R2": 10, "R3": 0, "R4": 0}

print("Initial:", R)
R["R3"] = R["R1"]          # R3 <- R1
print("R3 <- R1 :", R)

R["R4"] = R["R1"] + R["R2"] # ADD
print("R4 <- R1 + R2:", R)

R["R3"] = R["R1"] - R["R2"] # SUB
print("R3 <- R1 - R2:", R)

R["R4"] = R["R1"] & R["R2"] # AND
print("R4 <- R1 AND R2:", R)

R["R3"] = R["R1"] | R["R2"] # OR
print("R3 <- R1 OR R2:", R)
```

Sample Output

```
Initial: {'R1': 25, 'R2': 10, 'R3': 0, 'R4': 0}
R3 <- R1 : {'R1': 25, 'R2': 10, 'R3': 25, 'R4': 0}
R4 <- R1 + R2: {'R1': 25, 'R2': 10, 'R3': 25, 'R4': 35}
R3 <- R1 - R2: {'R1': 25, 'R2': 10, 'R3': 15, 'R4': 35}
R4 <- R1 AND R2: {'R1': 25, 'R2': 10, 'R3': 15, 'R4': 8}
R3 <- R1 OR R2: {'R1': 25, 'R2': 10, 'R3': 27, 'R4': 8}
```

Analysis

- R1 and R2 act as source registers and R3/R4 store transferred or computed results.
- The ALU receives operands from registers, performs the selected operation, and writes the result to a destination register.
- Register transfer instructions mainly move data, whereas ALU instructions transform data.
- For the example, 25 + 10 = 35, 25 − 10 = 15, 25 AND 10 = 8, and 25 OR 10 = 27.

Result: The simulation demonstrates how registers, control signals and the ALU cooperate to execute register transfer, arithmetic and logic operations.

2. Single-Bus and Multi-Bus Organization

Objective: Model data transfers between registers, memory and I/O using single-bus and multi-bus organizations and compare their transfer time and efficiency.

Theory

In a single-bus organization, only one major transfer can use the common bus at a time. This simplifies hardware but creates contention and serializes transfers. A multi-bus organization provides two or more independent data paths, allowing several transfers or operand reads to occur in parallel.

Simulation Model

```
# Each transfer is treated as one bus cycle.
operations = [
    "R1 <- Memory[100]",
    "R2 <- Memory[104]",
    "R3 <- R1 + R2",
    "Memory[108] <- R3",
    "IO <- R3"
]

single_bus_cycles = len(operations)

# Two independent buses allow compatible transfers to overlap.
multi_bus_cycles = 3

print("Operations:", len(operations))
print("Single-bus cycles:", single_bus_cycles)
print("Multi-bus cycles:", multi_bus_cycles)
print("Speedup:", round(single_bus_cycles / multi_bus_cycles, 2))
```

Sample Output

```
Operations: 5
```

Single-bus cycles: 5
Multi-bus cycles: 3
Speedup: 1.67

Comparison

Feature	Single Bus	Multi Bus	Effect
Data paths	One	Two or more	More parallelism
Transfer overlap	Limited	Possible	Higher throughput
Hardware	Simple	More complex	Higher cost
Bottleneck	High	Lower	Faster transfers
Example cycles	5	3	1.67× speedup

Analysis

The single-bus system is economical and easy to control, but every transfer competes for the same path. The multi-bus system uses additional paths to read operands and transfer data concurrently. Therefore, the multi-bus structure can reduce waiting time and improve CPU throughput, at the cost of additional hardware and control complexity.

Result: Multi-bus organization provides better transfer performance when parallel data movement is possible.

3. Five-Stage Instruction Pipeline

Objective: Simulate Fetch (F), Decode (D), Execute (E), Memory (M), and Write-back (W) stages and compare pipelined and non-pipelined execution.

Theory

A five-stage pipeline overlaps different instructions. While one instruction is in Execute, another can be in Decode and another in Fetch. For n instructions and k stages, an ideal pipeline requires approximately $k + n - 1$ cycles, whereas a non-pipelined processor requires approximately $n \times k$ cycles when each stage takes one cycle.

Python Simulation

```
instructions = ["I1", "I2", "I3", "I4", "I5", "I6"]
stages = ["F", "D", "E", "M", "W"]

n = len(instructions)
k = len(stages)

non_pipelined = n * k
pipelined = k + n - 1
speedup = non_pipelined / pipelined
throughput = n / pipelined
```

```
print("Instructions:", n)
print("Non-pipelined cycles:", non_pipelined)
print("Pipelined cycles:", pipelined)
print("Speedup:", round(speedup, 2))
print("Throughput:", round(throughput, 3), "instructions/cycle")

# Pipeline timing chart
for i, inst in enumerate(instructions):
    row = ["  "]*pipelined
    for s, stage in enumerate(stages):
    cycle = i + s
    if cycle < pipelined:
            row[cycle] = stage
    print(inst, " ".join(f"{x:>2}" for x in row))
```

Sample Output

```
Instructions: 6
Non-pipelined cycles: 30
Pipelined cycles: 10
Speedup: 3.0
Throughput: 0.6 instructions/cycle

I1  F D E M W
I2 F D E M W
I3          F D E M W
I4          F D E M W
I5          F D E M W
I6                  F D E M W
```

Analysis

- Non-pipelined execution completes one instruction before starting the next.
- Pipelining overlaps stages and significantly increases instruction throughput.
- For 6 instructions and 5 stages, ideal execution reduces 30 cycles to 10 cycles.
- Ideal speedup = 30/10 = 3.0. The theoretical maximum approaches 5 for a long instruction stream, but short programs and hazards reduce actual speedup.

Result: The pipeline improves throughput by overlapping instruction stages, although it does not reduce the latency of an individual instruction below the stage sequence.

4. Pipeline Hazards and Their Solutions

Objective: Demonstrate data, control and structural hazards and show how stalling, forwarding and branch prediction reduce their performance impact.

Types of Hazards

- Data hazard: A later instruction needs a result that an earlier instruction has not yet written. Example: ADD R1,R2,R3 followed by SUB R4,R1,R5.
- Control hazard: The next instruction depends on a branch decision. Example: BEQ R1,R2,LABEL.
- Structural hazard: Two stages need the same hardware resource in the same cycle, such as a single memory being used for both instruction fetch and data access.

Simple Hazard Simulation

```
instructions = [
    ("I1", "ADD R1,R2,R3"),
    ("I2", "SUB R4,R1,R5"),   # data dependency
    ("I3", "BEQ R4,R0,L"),          # control hazard
    ("I4", "LOAD R6,0(R7)")   # may conflict for a single memory
]

data_stall = 1        # forwarding reduces this dependency to one stall
branch_penalty = 2   # without prediction
structural_stall = 1 # single shared memory

base_cycles = len(instructions) + 4
hazard_cycles = base_cycles + data_stall + branch_penalty + structural_stall

# With forwarding, branch prediction and separate instruction/data paths:
improved_cycles = base_cycles + 0 + 0 + 0

print("Ideal cycles:", base_cycles)
print("With hazards:", hazard_cycles)
print("With mitigation:", improved_cycles)
print("Improvement:", hazard_cycles - improved_cycles, "cycles saved")
```

Sample Output

```
Ideal cycles: 8
With hazards: 12
With mitigation: 8
Improvement: 4 cycles saved
```

Hazard Resolution Techniques

Hazard	Problem	Technique
Data	Operand not ready	Data forwarding; stall if necessary
Control	Unknown next PC	Branch prediction; delayed branch; flushing
Structural	Resource conflict	Duplicate resources; separate I-cache/D-cache; scheduling

Analysis

Forwarding sends a produced value directly from a later pipeline stage to a dependent instruction without waiting for register write-back. Branch prediction guesses the next control-flow path, allowing the pipeline to continue; a wrong prediction causes a flush. Structural hazards can be reduced by providing independent resources or scheduling instructions so that conflicts do not occur.

Result: Hazard-handling techniques preserve pipeline utilization and reduce stalls, but incorrect branch predictions and unavoidable dependencies can still introduce penalties.

5. Superscalar, Out-of-Order, Speculative Execution and SMT

Objective: Analyze a processor that can issue multiple instructions per cycle and evaluate the effect of out-of-order execution, speculation and simultaneous multithreading (SMT) on throughput and resource utilization.

Theory

A superscalar processor has multiple execution units and can issue more than one independent instruction in a cycle. Out-of-order execution allows ready instructions to execute before older instructions that are waiting for operands. Speculative execution executes instructions before a branch outcome is known and discards incorrect work after a misprediction. SMT allows instructions from multiple hardware threads to share execution resources in the same cycle.

Simulation

```
# Simplified throughput model
cycles = 100
issue_width = 2

single_issue_instructions = cycles * 1
superscalar_instructions = cycles * issue_width

# Assume 75% average utilization because of dependencies, misses and hazards.
effective_utilization = 0.75
effective_instructions = superscalar_instructions * effective_utilization

# Two SMT threads help fill otherwise unused execution slots.
smt_utilization = 0.90
smt_instructions = cycles * issue_width * smt_utilization

print("Single issue:", int(single_issue_instructions), "instructions")
print("2-wide ideal:", int(superscalar_instructions), "instructions")
print("2-wide out-of-order effective:", int(effective_instructions), "instructions")
print("2-wide SMT effective:", int(smt_instructions), "instructions")
```

```
print("SMT vs single issue speedup:", round(smt_instructions/single_issue_instructions, 2))
```

Sample Output

```
Single issue: 100 instructions
2-wide ideal: 200 instructions
2-wide out-of-order effective: 150 instructions
2-wide SMT effective: 180 instructions
SMT vs single issue speedup: 1.8
```

Comparison

Technique	Main Idea	Benefit	Limitation
Superscalar	Issue multiple instructions	Higher throughput	Needs multiple execution units
Out-of-order	Execute ready instructions first	Hides stalls and latency	Complex scheduling hardware
Speculative	Execute predicted path	Reduces branch waiting	Wrong prediction wastes work
SMT	Mix instructions from threads	Improves resource utilization	Threads compete for resources

Analysis

A 2-wide superscalar processor has an ideal issue rate of two instructions per cycle. In practice, dependencies, branches, cache misses and resource conflicts reduce this rate. Out-of-order execution improves utilization by selecting ready instructions, while speculation reduces control-flow waiting. SMT can fill unused execution slots with instructions from another thread, increasing overall throughput. The numerical results are an illustrative simulation; actual processor performance depends on workload and microarchitecture.

Result: Superscalar execution, out-of-order scheduling, speculation and SMT work together to increase instruction-level and thread-level parallelism.

Overall Conclusion

The five simulations demonstrate the progression from basic register-level data movement to advanced processor parallelism. Register transfers and ALU operations form the basic datapath activities. Single-bus and multi-bus organization show how datapath structure affects transfer efficiency. Pipelining increases instruction throughput by overlapping stages, while hazard-handling mechanisms maintain correct execution. Finally, superscalar, out-of-order, speculative execution and SMT exploit instruction-level and thread-level parallelism to improve resource utilization and throughput.

Suggested References

- M. Morris Mano, Computer System Architecture, Pearson.
- Carl Hamacher, Safwat Zaky and V. Carl, Computer Organization.
- David A. Patterson and John L. Hennessy, Computer Organization and Design.